

Step 4: AgentCore Observability

[AgentCore Observability](#) equips the Runtime, Memory, and Gateway services with monitoring and tracing capabilities. Observability data is generated using Amazon OpenTelemetry instrumentation and is integrated with Amazon CloudWatch and AWS CloudTrail. These integrations provide comprehensive visibility into agent applications, enabling detailed tracing, debugging, and monitoring of AI agent performance in production environments. Engineering and operations teams can use these features to observe operational metrics, detect issues, and maintain the quality and reliability of deployed AI systems.

This foundation enables layers of production observability:

1. **GenAI Observability Dashboard** – CloudWatch’s purpose-built view for AI agents with real-time metrics, session analytics, trace visualizations, and error breakdowns for rapid diagnosis and optimization.
2. **CloudWatch Metrics** – Automatically published numerical indicators (invocations, latency, duration, sessions, error rates) for Runtime, Memory, and Gateway that power dashboards, alerts, and trend analysis.
3. **CloudWatch Logs** – Structured, time-stamped application and system logs (including span/log records) used for deep troubleshooting and forensic analysis.
4. **AWS CloudTrail** – Immutable audit trail of management and data-plane API activity (who did what, when, and from where) for security monitoring and compliance.

Now that your agent is deployed to AgentCore Runtime and integrated with Memory and Gateway, let's explore each observability layer in detail.

4.1 Understanding AgentCore's Built-in Observability

AgentCore provides comprehensive built-in observability to help monitor, debug, and optimize AI agents in production environments. Using CloudWatch’s dedicated GenAI Observability dashboard, developers and operators can access layered insights about agent activities, session behavior, and operational traces—all with minimal configuration.

Agents

By default, AgentCore forwards agent traces to **CloudWatch GenAI Observability**. To view them, open CloudWatch → GenAI Observability → Bedrock AgentCore. From here you can review agent activity, drill into sessions, and open traces for detailed analysis.

 Agents Overview on CloudWatch

The Agents view provides a high-level performance overview across all deployed agents:

- **Session distribution** by agent and time period
- **Error rate visualization** and performance trends
- **Agent comparison** for multi-agent deployments
- **Real-time activity monitoring** and usage patterns

Sessions

The Sessions view shows the list of all the sessions associated with all agents in your account, with filters and drill-downs that make it easy to investigate behavior for a specific session.



Use the Sessions view to:

- **Find and filter sessions** by session ID, agent name, status, duration, or time range to quickly locate the conversation you care about.
- **Open a session detail dashboard** by clicking a row to see per-session metrics (latency, errors, span count), message timeline, and related resources.
- **Trace a conversation end-to-end** by jumping from the session to its associated traces and spans, preserving context with `sessionId` and `traceId` correlation.
- **Compare behavior across sessions** to spot outliers (e.g., unusually high latency, repeated tool failures, or long conversations).

What you learn from Sessions:

- **Lifecycle and outcome** – when the session started/ended, how long it lasted, and whether any errors occurred.
- **Engagement patterns** – number of turns, gaps between messages, and overall conversation depth.
- **Context usage** – whether Memory was accessed (e.g., `RetrieveMemoryRecords` , `CreateEvent`) during the session.
- **Tool utilization** – which Gateway tools were invoked and their contribution to total time (via linked traces).

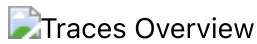
Tips:

- Start here when you have a user-reported issue; filter by timeframe and scan for abnormal duration or error indicators, then drill into traces.
- Use the same time range across Sessions, Traces, and Logs to keep investigations aligned.

Traces

The Traces view lists all request traces emitted by your agents. Use it to find, analyze, and export end-to-end executions:

- Use **Filter traces** to search by time range, duration, status, or attributes.
- **Sort columns** (e.g., duration, start time) to prioritize what needs attention.
- From **Actions**, open **Logs Insights** to correlate spans with structured logs, or **Export selected traces** for offline analysis.



Working with Traces:

- **Filter by latency patterns** - Find all traces exceeding SLA thresholds
- **Error correlation analysis** - Identify patterns in failed requests
- **Component bottleneck identification** - Determine which spans contribute most to latency
- **Export selected traces** for offline analysis and reporting

4.2 AgentCore Observability Metrics

AgentCore automatically publishes **built-in observability metrics** for all primitives Runtime, Memory, Gateway, and Tools to **Amazon CloudWatch** under the namespace `AWS/Bedrock-AgentCore`. These metrics give you real-time visibility into request activity, latency, error rates, session usage, and performance. You can use them to build dashboards, configure alarms, and analyze trends for production operations.

Navigating to AgentCore Metrics in CloudWatch

1. Open CloudWatch Console

- Navigate to the **CloudWatch** service in the AWS Console.
- Select **Metrics** from the left navigation panel.
- Click **All metrics**.

2. Locate AgentCore Metrics

- In the metrics browser, select the namespace **AWS/Bedrock-AgentCore**.
- Explore the available dimension combinations for each resource type.

3. Filter by Resource

- Use the search bar to enter the ARN or name of your resource.
- Examples: `customer_support_agent`, `CustomerSupportMemory`, or `customersupport-gw`.

Runtime Metrics

Runtime metrics capture how your agent handles requests in production and how users experience responsiveness.

Key Metrics to Monitor

- **Invocations** – Total requests received by the agent, showing usage and peak load periods.
- **Latency** – End-to-end response time (request to final token), including model inference, memory lookups, and tool execution.
- **Sessions** – Number of active agent sessions, useful for capacity planning and engagement analysis.
- **Error Metrics:**
 - **User Errors** – Invalid requests (400), missing resources (404), or permission errors (403).
 - **System Errors** – Internal service errors (500) that may indicate infrastructure issues.
 - **Throttles** – Requests rejected due to TPS or quota limits (429).

Memory Metrics

Memory metrics measure how efficiently the agent stores and retrieves conversation context, and how background processing maintains memory quality over time.

Key Metrics to Monitor

- **Latency** – End-to-end processing time for memory operations.
- **Invocations** – Total number of API requests to the Memory service.
- **System Errors** – Memory API calls that failed with AWS server-side errors (5xx).
- **User Errors** – Memory API calls that failed with client-side errors (4xx).
- **Errors** – Total errors across control-plane and data-plane operations, including ingestion failures.
- **Throttles** – Requests throttled (429), not counted as invocations or errors.
- **Creation Count** – Number of new memory events and records created.

Gateway Metrics

Gateway metrics provide visibility into tool execution performance, MCP protocol health, and target distribution.

Key Metrics to Monitor

- **Invocations** – Total requests made to Gateway Data Plane APIs.
- **Throttles (429)** – Requests throttled due to exceeded limits.
- **System Errors (5xx)** – Requests that failed due to server-side issues.
- **User Errors (4xx)** – Requests that failed due to client errors (excluding throttles).
- **Latency** – Time from receiving a request until the first response token is sent.
- **Duration** – Full end-to-end request time until the final response token is sent.
- **TargetExecutionTime** – Time taken by the target (Lambda, API) to execute, excluding Gateway overhead.

- **TargetType** – Distribution of requests served by target type (MCP, Lambda, OpenAPI).

Best Practices for Metrics

- **Dashboards** – Combine Runtime, Memory, and Gateway metrics in a single CloudWatch dashboard.
- **Alarms** – Configure CloudWatch Alarms for high latency, error rates, or throttling.
- **Trends** – Monitor growth in `CreationCount` and `Sessions` to anticipate scaling needs.
- **Correlation** – Combine metrics with spans and logs for deep troubleshooting.

With these metrics in CloudWatch, you have baseline observability across AgentCore resources, enabling real-time monitoring and proactive alerting.

4.3 AgentCore Observability Spans

In addition to metrics, AgentCore can provide **structured spans** that capture fine-grained details about specific operations. Spans allow you to trace execution paths, measure operation latency, and debug failures in depth.

Memory Spans

AgentCore Memory supports spans for the following operations (when enabled):

- **CreateEvent** – New memory event creation.
- **GetEvent** – Retrieval of an existing memory event.
- **ListEvents** – Listing all events within a session.
- **DeleteEvent** – Deletion of a memory event.
- **RetrieveMemoryRecords** – Retrieval of memory records for a namespace.
- **ListMemoryRecords** – Listing available memory records.

Each span includes attributes such as `memory.id`, `session.id`, `event.id`, `actor.id`, and flags (`throttled`, `error`, `fault`).

Viewing Spans

- **CloudWatch Logs** – Spans are stored as structured logs under `/aws/vendedlogs/bedrock-agentcore/memory/<memory_id>`.
 - **CloudWatch Application Signals** – Provides visualizations and end-to-end trace analysis.
1. Open the **CloudWatch Console**.
 2. Navigate to **Logs** → **Log groups**.
 3. Find the log group under `/aws/vendedlogs/bedrock-agentcore/memory/<memory_id>`.

4. Use **CloudWatch Application Signals** (GenAI Observability) for trace visualizations and span metrics.

Screenshot Placeholder: CloudWatch Application Signals span view

Use Cases for Spans

- Identify bottlenecks in memory retrieval or event creation.
- Debug failures in consolidation or extraction processes.
- Correlate spans with metrics and logs for a full observability picture.

By combining **metrics** (high-level monitoring) and **spans** (operation-level tracing), you gain comprehensive visibility into the behavior and performance of your AgentCore resources.

4.4 AgentCore Observability Logs

CloudWatch Logs capture detailed, time-stamped events across AgentCore components. These structured JSON logs provide visibility into agent operations, background processing, and system behavior for deep troubleshooting and performance analysis.

Now that you have built a customer support agent system across Labs 1-4 with local tools (Lab 1), enhancing it with persistent memory for conversation context and personalization (Lab 2), integrating with AgentCore Gateway to centrally manage shared tools and secure authentication (Lab 3), and deploying to AgentCore Runtime for production-ready auto-scaling with observability (Lab 4)—let's explore the detailed logs each component generates to help you monitor and debug your production agent system.

Memory Logs - Setup and Navigation

Memory logs capture the background processing operations that occur when your customer support agent (from Lab 1) stores and retrieves conversation context using the Memory resource you created in Lab 2. These logs provide deep insight into how AgentCore extracts, consolidates, and persists customer preferences and conversation context.

Enable Memory Log Delivery:

1. **Navigate to your Memory resource** from Lab 2:
 - Go to Amazon Bedrock AgentCore → Memory → **CustomerSupportMemory**
2. **Configure Log Delivery:**
 - In the **Observability** section, find **Log delivery**
 - Click **Add** → Select **Log type:** APPLICATION_LOGS

- **Destination log group:** /aws/vendedlogs/bedrock-agentcore/memory/APPLICATION_LOGS/CustomerSupportMemory-xxxxxxxxxxxx

The log group name includes an automatically generated ID suffix (like CustomerSupportMemory-WcEhTTFp10) unique to your Memory resource.

3. Access Memory Logs in CloudWatch:

- Navigate to **CloudWatch** → **Logs** → **Log groups**
- Find: /aws/vendedlogs/bedrock-agentcore/memory/APPLICATION_LOGS/CustomerSupportMemory-[auto-generated-ID]
- Click to open: **BedrockAgentCoreMemory_ApplicationLogs**

Understanding Memory Processing

When customers interact with your agent, AgentCore Memory processes conversations through a **three-phase pipeline**:

1. **Extraction** → Convert conversation events to structured memories
2. **Consolidation** → Merge new memories with existing ones to avoid duplication
3. **Storage** → Persist consolidated memories to the memory store

Each phase generates detailed logs that help you monitor performance, debug issues, and understand memory behavior.

Memory Log Structure and Analysis

Sample Memory Log Entry:

json

```
{
  "resource_arn": "arn:aws:bedrock-agentcore:<region>:<account-id>:memory/Cu
  "event_timestamp": 1756957336547,
  "memory_strategy_id": "CustomerPreferences-EH93NK8WlQ",
  "namespace": "support/customer/customer_001/preferences",
  "actor_id": "customer_001",
  "session_id": "7e68e007-8675-464d-864b-db6c97b2a078",
  "body": {
    "log": "Processing extraction input",
    "requestId": "58bc9c8c-4e84-40b6-bcf3-1161866443bb",
    "isError": false
  }
}
```

Key Fields Explained:

Field	Description
resource_arn	ARN of the Memory resource—useful for filtering and distinguishing events across multiple memory resources.
event_timestamp	Timestamp (in milliseconds) indicating when the memory operation occurred.
memory_strategy_id	Identifier of the memory strategy (e.g., <code>CustomerPreferences</code> , <code>CustomerSupportSemantic</code>)—lets you distinguish processing logic.
namespace	Logical path (such as <code>support/customer/{customer_id}/{type}</code>) that organizes and separates memory data for each context, user, session, or memory type.
actor_id	Identifier of the user or agent actor (e.g., <code>customer_001</code>) that helps attribute operations to individuals or systems.
session_id	Identifier linking memory operations to a specific conversation session, enabling end-to-end traceability.
body.log	Human-readable log message describing the current operation step—useful during extraction or consolidation.
body.requestId	Unique identifier correlating related memory operations—useful for tracing a single memory workflow.
body.isError	Boolean flag (<code>true</code> or <code>false</code>) indicating whether the operation encountered an error.

Complete Memory Processing Example

Let's trace a real memory processing workflow. When `customer_001` interacts with your agent about laptop preferences, you'll see this complete sequence in CloudWatch Logs:

json



```
{"log": "Processing extraction input"}
{"log": "Starting to process Preference strategies."}
{"log": "Extraction completed in 1674 ms"}
{"log": "Extracted 1 memories"}
{"log": "Processing consolidation input"}
{"log": "Retrieving memories."}
{"log": "Succeeded to retrieve 2 records."}
{"log": "1 memories require consolidation."}
{"log": "Consolidating 2 facts with 3 related memories"}
{"log": "Succeeded to upsert 1 records."}
{"log": "Succeeded operation for record id mem-af46dbb9..."}
```

What This Sequence Reveals:

1. Extraction Timing

- The log shows Extraction completed in 1674 ms .
- That's ~1.6s latency for converting the raw conversation into structured memory.
- If you consistently saw higher numbers here (e.g., >3000 ms), you'd flag this as a potential bottleneck.

2. New Memory Created

- Extracted 1 memories confirms only one new record was generated for this interaction.
- This tells you the preference strategy was triggered correctly and produced new data.

3. Consolidation Behavior

- Logs show Succeeded to retrieve 2 records. and 1 memories require consolidation.
- This means the system correctly pulled existing records and determined overlap, then merged facts (Consolidating 2 facts with 3 related memories).
- This is critical for **avoiding duplication** and ensuring the agent doesn't keep multiple conflicting memories for the same customer.

4. Successful Storage

- Succeeded to upsert 1 records. followed by a record ID confirms the memory store was updated.
- That's the final confirmation the lifecycle completed successfully.

What This Observation Tells You

- **Performance:** Extraction + consolidation pipeline took ~1.6s, which is reasonable but worth monitoring under load.
- **Correctness:** Strategies (`CustomerPreferences`) are firing as expected, producing new memory records.
- **Data Integrity:** Consolidation ensures that the memory store stays consistent and avoids bloat.
- **Traceability:** Because you have the `session_id` and `requestId` , you can trace this specific customer's memory lifecycle end-to-end.

Gateway Logs - Setup and Navigation

Gateway logs capture the MCP (Model Context Protocol) operations that occur when your customer support agent (from Lab 1) uses the centralized tools you configured in Lab 3. These logs provide detailed insight into how AgentCore Gateway manages tool discovery, execution, and performance for the `check_warranty_status` , `web_search` , and `get_product_info` tools.

Enable Gateway Log Delivery:

1. Navigate to your Gateway resource from Lab 3:

- Go to Amazon Bedrock AgentCore → Gateways → **customersupport-gw**

2. Configure Log Delivery:

- In the **Observability** section, find **Log delivery**
- Click **Add** → Select **Log type:** `APPLICATION_LOGS`
- **Destination log group:** `/aws/vendedlogs/bedrock-agentcore/gateway/APPLICATION_LOGS/customersupport-gw-xxxxxxxxxxxx`

The log group name includes an automatically generated ID suffix (like `customersupport-gw-dcbgswzb5p`) unique to your Gateway resource.

3. Access Gateway Logs in CloudWatch:

- Navigate to **CloudWatch** → **Logs** → **Log groups**
- Find: `/aws/vendedlogs/bedrock-agentcore/gateway/APPLICATION_LOGS/customersupport-gw-[auto-generated-ID]`
- Click to open: **BedrockAgentCoreGateway_ApplicationLogs**

Understanding Gateway Processing

When customers interact with your agent, AgentCore Gateway processes tool requests through a **four-phase MCP protocol**:

1. **Initialization** → Establish MCP connection with the gateway
2. **Discovery** → Enumerate available tools for the agent
3. **Execution** → Invoke specific tools (warranty check, web search, etc.)
4. **Correlation** → Track multiple tool calls within the same session

Each phase generates detailed logs that help you monitor tool performance, debug MCP issues, and understand tool usage patterns.

Gateway Log Structure and Analysis

Sample Gateway Log Entry:

```
json
{
  "resource_arn": "arn:aws:bedrock-agentcore:<region>:<account-id>:gateway/c
  "event_timestamp": 1756958992250,
  "body": {
    "id": "2",
    "log": "Received request for tools/call method",
    "isError": false
  },
  "account_id": "<account-id>",
  "request_id": "b2474cdc-b5dd-44b0-876d-8de900414ee3"
}
```

Key Fields Explained:

Field	Description
resource_arn	ARN of the Gateway resource—useful for filtering logs across multiple gateway instances.
event_timestamp	Timestamp (in milliseconds) indicating when the gateway operation occurred.
body.id	Sequential request identifier within the gateway session for tracking operation order.
body.log	Human-readable operation description detailing MCP protocol interactions.
body.isError	Boolean flag (<code>true</code> or <code>false</code>) indicating whether the operation encountered an error.
account_id	AWS account identifier for multi-account deployments.
request_id	Unique correlation ID linking gateway operations to agent requests—enables end-to-end tracing.

Complete Gateway Processing Example

Let's trace a real tool execution workflow. When a customer asks your agent "Can you check the warranty on my laptop?", you'll see this complete MCP sequence in CloudWatch Logs:

```
json
{"log": "Started processing request with requestId: 0"}
{"log": "Received request for initialize method"}
{"log": "Successfully processed request with requestId: 0"}
{"log": "Started processing request with requestId: 1"}
{"log": "Received request for tools/list method"}
{"log": "Successfully processed request with requestId: 1"}
{"log": "Started processing request with requestId: 2"}
{"log": "Received request for tools/call method"}
{"log": "Executing tool LambdaUsingSDK__check_warranty_status from target M"}
{"log": "Successfully processed request with requestId: 2"}
```

What This Sequence Reveals:

1. MCP Initialization

- Received request for `initialize` method confirms proper gateway connection

- Request ID 0 shows this is the first operation in the session
- Successful completion enables subsequent tool operations

2. Tool Discovery

- Received request for tools/list method shows agent querying available tools
- Gateway provides the list of Lab 3 tools: `check_warranty_status` , `web_search` , `get_product_info`
- Request ID 1 maintains proper sequencing

3. Tool Execution

- Received request for tools/call method indicates specific tool invocation
- Executing tool `LambdaUsingSDK__check_warranty_status` shows which Lab 3 tool is running
- from target `ME2UI4BINR` identifies the specific Lambda function you deployed

4. Performance Tracking

- Request IDs (0→1→2) show proper sequential processing
- Timestamp analysis reveals tool execution latency
- Successfully processed confirms no errors in the MCP protocol

What This Observation Tells You

- **Protocol Health:** MCP initialization and discovery working correctly for your Lab 3 setup
- **Tool Availability:** All workshop tools are properly registered and discoverable
- **Execution Success:** Customer warranty requests are successfully routed to your Lambda function
- **Traceability:** Complete request correlation from agent query to tool execution

Practical Gateway Log Use Cases

1. Tool Performance Monitoring

plaintext



Target: Identify slow-executing tools in your workshop deployment
Monitor: Tool execution timing, Lambda cold starts, timeout patterns
Alert: When tools consistently >2000ms or show increasing latency trends

2. MCP Protocol Debugging

plaintext



```
Target: Verify Lab 3 gateway configuration is working correctly  
Monitor: Initialization success, tool discovery completion, execution pattern  
Debug: Failed tool calls, missing tools, or protocol errors
```

3. Workshop Tool Usage Analytics

plaintext



```
Target: Understand which Lab 3 tools customers use most frequently  
Monitor: check_warranty_status vs web_search vs get_product_info usage  
Optimize: Tool caching strategies and Lambda concurrency settings
```